Systems Programming in C

Study notes on SPC7 and SPC8

Covers every slide in **SPC7: Pointer Arithmetic & Structures** (35 slides) and **SPC8: Linked Lists, Enums, and Function Pointers** (22 slides). Slide references look like [7:12] = SPC7 slide 12, and [8:6] = SPC8 slide 6. Corrections to typos on the slides and extra explanations that are *not* on the slides are put in amber boxes so you can tell them apart.

Part	Topic	Slides
1	Pointers are addresses; pointer addition, subtraction, comparison, casting	7:2-7, 7:26-32
2	Arrays and pointers (decay, equivalences, differences)	7:8-11, 7:20
3	Structures, typedef, struct operations, self-referential structs	7:12-17, 7:23-25, 7:35
4	Alignment and padding	7:18, 7:33-34
5	Pointer casting, returning several values	7:21-22
6	Linked lists (node, head, create, prepend, find last, append)	8:2-10
7	Enumeration types	8:11-12
8	The const qualifier	8:13
9	Function pointers and qsort	8:14-22
10	Cheat sheet, common bugs, self-check questions	-

> **The big picture**
>
> Everything in these two decks hangs together on one idea: **a pointer is just a number (a byte address), and the compiler uses the pointer's type to decide how to scale and interpret it.** Arrays, structs, linked lists, qsort and function pointers are all different ways of using that one idea.

Part 1 - Pointer arithmetic

1.1 Pointers are addresses [7:2]

- The value of a pointer is a **byte address**: an unsigned integer used to number the bytes of memory.
- Range of valid values:
  - `0x00000000` to `0xFFFFFFFF` on a 32-bit system
  - `0x0000000000000000` to `0xFFFFFFFFFFFFFFFF` on a 64-bit system
- **Corollary:** if a pointer is an integer, you can do arithmetic on it to compute other addresses.

```
int *p = malloc(sizeof(int) * 10);
int *q = p + 1;      // q is the address of the SECOND int, not p + 1 byte
```

A pointer is 4 bytes wide on a 32-bit machine and 8 bytes wide on a 64-bit machine. That is why `sizeof(p)` is 8 in the examples on [7:10] and [7:11] (the slides assume a 64-bit system).

1.2 Adding an integer to a pointer [7:3-4]

> **Key idea**
>
> If `p` has type `T *` and `k` is an integer, then both `p + k` and `k + p` are valid and have type `T *`. The integer is **automatically scaled by the size of the type pointed to**.

```
byte address of (p + k)  =  (unsigned long)(address stored in p) + k * sizeof(T)
```

Slide 3 example. Suppose `p` is an `int *` whose value is 1000 and `sizeof(int)` is 4. Then `p + 1` is **1004**, not 1001: it is the address of the next *item* of type `int`.

Address	Pointer expression	Two ways to read the value
1020	p + 5	`*(p+5)` or `p[5]`
1016	p + 4	`*(p+4)` or `p[4]`
1012	p + 3	`*(p+3)` or `p[3]`
1008	p + 2	`*(p+2)` or `p[2]`
1004	p + 1	`*(p+1)` or `p[1]`
1000	p (p = 1000)	`*p` or `p[0]`
996	p - 1	`*(p-1)` or `p[-1]`
992	p - 2	`*(p-2)` or `p[-2]`

- Subtraction of an integer scales the same way: `p - 2` is `1000 - 2*4 = 992`.
- `p[i]` is just a nicer way to write `*(p+i)`. Negative indexes are legal as long as the address is still inside memory you own.
- `void *` **arithmetic:** the C standard does **not** allow it (the compiler cannot scale by the size of `void`). gcc has an *extension* that treats `sizeof(void)` as 1, so it works in gcc but is not portable.

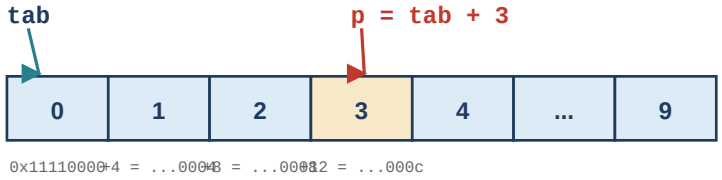
1.3 Worked example: tab + 3 [7:26-32]

These slides build one example step by step. Read the code, then trace it on the diagram.

Slides 26-29 (same code, the diagram changes)

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *tab = (int*)malloc(sizeof(int)*10);   // room for 10 ints
    tab[3]   = 10;                             // write 10 into element 3
    int *p   = tab + 3;                        // p points at element 3
    printf("What is at tab+3? = %d\n", *p);    // prints 10
    *p = 20;                                   // changes tab[3] through p
    printf("What is at tab[3]? = %d\n", tab[3]);   // prints 20
    return 0;
}
```



- [7:26] `tab` points to a block of 10 ints (indexes 0..9).
- [7:27] `tab[3] = 10` stores 10 in element 3.
- [7:28] `p = tab + 3` makes `p` point at that same element. `*p` is 10.
- [7:29] `*p = 20` writes through the pointer, so `tab[3]` is now 20. `p` and `tab[3]` name the same memory.

What about the actual memory addresses? [7:30-32] The slides ask: if `tab` is `0x11110000`, is `tab+1` equal to `0x11110001`? **No.** It is `0x11110004`, because each `int` is 4 bytes (the "WHY?" picture). The bottom-line slide spells out the whole computation:

```
p = tab + 3 = tab + 3 * sizeof(int) = tab + 3 * 4 = 0x11110000 + 12 = 0x1111000c
```

> **Bottom line [7:32]**
>
> The offset (3) is **scaled by the compiler** with the size of the type to get an address in bytes. You write element counts; the compiler generates byte offsets.

> **Beyond the slides (added for understanding)**
>
> The slide code uses curly quotes in `printf` and omits `#include <stdio.h>`; use straight quotes and include `<stdio.h>` or it will not compile. Also note that 12 in decimal is `0xc` in hex, so `0x11110000 + 12 = 0x1111000c`.

1.4 Pointer subtraction [7:5]

- You can subtract one pointer from another **only if both have the same type**.
- The result is the **number of data items between them, not the number of bytes**.

ptrsub.c from the slide

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p = malloc(sizeof(int)*10);
    int *last = p + 9;
    int  dist = last - p;          // both are int *
    printf("Distance is %d\n", dist);
    free(p);
    return 0;
}
// Output:  Distance is 9
```

Here the byte difference is 36, but the answer printed is 9, because the compiler divides by `sizeof(int)`. Subtraction is the inverse of addition: `(p + k) - p == k`.

Beyond the slides (added for understanding)

The true type of a pointer difference is `ptrdiff_t` (from `<stddef.h>`), printed with `%td`. The slide stores it in an `int`, which is fine for small distances. Subtracting pointers that are not inside the same array is not meaningful.

1.5 Pointer comparisons [7:6]

- All six comparison operators work on pointers: `< > <= >= != ==`.
- **Purposes:**
  - check boundary conditions in arrays
  - manually manage memory blocks
- **Semantics:** purely based on memory layout. The bits of the two pointers are compared **as unsigned integers**. A pointer with a lower address is "less than" one with a higher address.
- Typical use is a loop that runs until a pointer reaches a limit:

```
while (b < a) {
    // do something
    b++;               // (the slide shows only the condition; b must move toward a)
}
```

In the slide's picture, memory runs from `0x00000000` at the bottom to `0xffffffff` at the top, with `b` at a lower address than `a`, so `b < a` is true until `b` catches up.

Watch out

Comparing pointers of **different types** gives a compiler warning: `warning: comparison of distinct pointer types lacks a cast` (from the slide notes). Cast one of them if you really mean it.

1.6 Effect of casting on pointer arithmetic [7:7]

- If you cast a pointer, **all arithmetic after the cast uses the type you cast to**.
- To get **unscaled** (byte-by-byte) arithmetic, cast to `(char *)` because `sizeof(char)` is 1.

```
int  *t;
char *p = (char *) t + 8;      // cast first, then add: 8 is NOT scaled -> t + 8 bytes
char *q = (char *) (t + 8);    // add first (scaled by 4), then cast   -> t + 32 bytes
```

Expression	Order of operations	Byte offset from t
`(char *) t + 8`	cast binds first, then `+ 8` on a `char *`	8
`(char *) (t + 8)`	`t + 8` on an `int *` (scaled by 4), then cast	32

The slide notes put it simply: the scaling constants are different (1 versus 4).

1.7 Typecasting pointers and "weak typing" [7:21]

- C lets you cast pointers in **any** way you like, so you can "forge" pointers that point wherever you wish.
- This permissiveness is called **weak typing**.

```
float f;
char  p  = * (char *) &f;      // the 1st byte of the representation of f
char  ch = * (char *) 1000;    // read ANY byte in memory (address 1000)
```

- `&f` is a `float *`; casting it to `char *` lets you look at the float one byte at a time.
- `(char *)1000` turns the integer 1000 into an address. Dereferencing it reads whatever is at that address. Doing this on an address you do not own normally crashes (segmentation fault).
- The slide's picture caption ("That's what makes C very attractive for...") points at low-level systems work, where you really do need to touch specific addresses.

Part 2 - Arrays and pointers

2.1 They can often be used interchangeably [7:8]

```
int  a[10];
int *p = a;

// all of these are the value of a[0]:     *p     p[0]     *a     a[0]
// all of these are the value of a[1]:     *(p+1) p[1]     *(a+1) a[1]
// all of these are the ADDRESS of a[0]    p      &p[0]    a      &a[0]      (type is int *)
```

> **Key idea**
>
> **Array decay:** in most expressions the name of an array "becomes" a pointer to its first element: an expression of type "array of int" is converted to "pointer to int".

The exceptions (from the C99 standard quote in the slide notes, section 6.3.2.1/3). An array expression is NOT converted to a pointer when it is:

- the operand of `sizeof`
- the operand of the unary `&`
- a string literal used to initialise an array

Also from the notes: after decay the result is **not an lvalue**, so you cannot operate on the array itself (for example, assign to it or increment it), but you can operate on a pointer.

2.2 Example: five ways to initialise an array [7:9]

```
int a[10], *p = a;   // note: this is  int *p; p = a;   (NOT  *p = a)

for (int i=0; i<10; i++) a[i]      = i;   // array indexing
for (int i=0; i<10; i++) p[i]      = i;   // indexing via a pointer
for (int i=0; i<10; i++) *(p+i)    = i;   // explicit pointer arithmetic
for (    i=0; i<10; i++) i[p]      = i;   // obfuscated but valid C!
for (    i=0; i<10; i++) *p++      = i;   // the common pointer idiom
```

- `int a[10], *p = a;` declares both. The `*` belongs to `p` and the initialiser `= a` is assigned to `p`, not to `*p`.
- `i[p]` is valid because `p[i]` means `*(p+i)` and addition is commutative: `*(p+i) == *(i+p) == i[p]`.
- `*p++ = i;` is the classic idiom. It is equivalent to the three-step sequence on the slide:

```
int *tp = p;     // remember the old address
p++;             // advance p to the next element
*tp = i;         // store through the OLD address
```

`*p++` parses as `*(p++)`: the value of `p++` is the old pointer, and `p` itself moves ahead one element. Use it to walk through an array in one expression.

The same idiom appears in the **array-copy** example [7:20]:

```
// using array indexing
void copy_array0(int source[], int target[], int n) {
    for (int i = 0; i < n; i++)
        target[i] = source[i];
}

// using pointers
void copy_array1(int source[], int target[], int n) {
    for (int i = 0; i < n; i++)
        *target++ = *source++;
}
/* *target++ = *source++;  is the same as:
       int *tp = source;  source++;  *target = *tp;  target++;   */
```

Inside the function, `source` and `target` are really `int *` (see 2.4), so they can be incremented.

2.3 Arrays and pointers are NOT the same [7:10-11]

Even though they often behave alike, they are different things:

```
int  a[10];
int *p = a;        // a is converted to int *

&a        // pointer to an array of 10 ints         type  int (*)[10]
&p        // pointer to a pointer to int            type  int **

sizeof(a) // 40, because a is an array of 10 ints (10 * 4)
sizeof(p) // 8,  because p is a pointer (64-bit system)

p++;      // OK: p is a variable that can change
a++;      // ERROR: will not compile.  Like  2++  (you cannot modify a constant)
```

	Array `a`	Pointer `p`
What it is	A block of 10 ints, a fixed place in memory	A variable holding an address
`sizeof`	40 (whole array)	8 (just the address, on 64-bit)
`&` gives	`int (*)[10]`	`int **`
Can be incremented?	No (not a modifiable lvalue)	Yes

Example [7:11]:

```c
#include <stdio.h>

void foo(int *x)
{
    printf("%lu\n", sizeof(x));
}

int main()
{
    int a[10], *p;

    p = a;                                          // a is converted to int *
    printf("%lu %lu\n", sizeof(a), sizeof(p));    // a is still an array in sizeof
    foo(a);                                         // a is converted to int *
}
/* Output:   40 8
             8                                                          */
```

- First line `40 8`: `sizeof(a)` = 40 (array), `sizeof(p)` = 8 (pointer).
- Second line `8`: when `a` is passed to `foo`, it decays to `int *`, so inside `foo`, `sizeof(x)` is the size of a pointer, not of the array.

> **Watch out**
>
> A function can never learn the length of an array from the pointer it receives. Always pass the length separately (as `n` in `copy_array0`).
>
> The slide note says you cannot write `int a[]` for the array in `main` here: a local array needs a size or an initialiser. (As a function *parameter*, `int x[]` is allowed and means `int *x`.)

2.4 Structures, built on the same ideas

Structures continue in Part 3. First, a recap of the part you just covered:

> **Recap**
>
> `a[i]` is `*(a+i)`. The compiler scales `i` by `sizeof(element)`.
>
> An array name decays to a pointer to element 0 (except under `sizeof`, `&`, and string-literal initialisation).
>
> Arrays are not pointers: sizes, `&`, and incrementing all differ.
>
> Pointer difference counts elements; pointer comparison compares addresses as unsigned numbers.

Part 3 - Structures and typedef

3.1 What a structure is [7:12-14]

- A mechanism to define **new types**, also called **compound types**.
- Used to **aggregate related variables of different types** into one unit.
- A structure type declaration gives the type a name and a set of **members**. Members may be:
  - basic types
  - pointers
  - arrays
  - other structures

- A structure *variable definition* names a variable of that type, and **allocates the memory for it but does not initialise it** (slide note).

Slide 12

```
struct student_grade {        // type declaration
    char *name;
    int   id;
    char  grade[3];
};
...
struct student_grade  student1;                    // one variable
struct student_grade  student2, student3;          // several
struct student_grade  all[200];                    // array of 200 structs
```

Simple example [7:13] and the field access syntax (same dot syntax as Java or Python):

```
struct Person {          // structure type declaration
    int  age;
    char gender;
};

int main() {
    struct Person p;             // variable definition
    p.age    = 44;               // field access with .
    p.gender = 'M';

    struct Person q = {44, 'M'}; // definition AND initialisation (members in order)
    return 0;
}
```

Array of structures with nested initialisers [7:14]:

```
struct Person {
    char name[32];
    int  age;
    char gender;
};

int main()
{
    struct Person family[4] = {          // four persons, indexed 0..3
        {"Alice",   34, 'F'},
        {"Bob",     40, 'M'},
        {"Charles", 15, 'M'},
        {"David",   13, 'M'}
    };
    int juniorAge = family[3].age;       // David's age: 13
    return 0;
}
```

- The member `name` is a `char` array, so each name can be **at most 31 characters** (the 32nd byte is the `'\0'` terminator).
- `family[3].age` combines indexing (`[3]` picks the struct) and field access (`.age`).

3.2 typedef [7:15, 7:23]

- Struct names get long (`struct Person` every time). `typedef` gives an **existing type a new name**, which makes code more readable.
- It is very common to **combine** the struct declaration and the typedef.

```
struct Person { char name[32]; int age; char gender; };
typedef struct Person TPerson;          // two steps

typedef struct Person {                 // one step: declaration + typedef
    char name[32];
    int  age;
    char gender;
} TPerson;

TPerson family[4];                      // no 'struct' keyword needed now
```

Reading typedefs [7:23]. The slide's hint: *think about how you would define a variable*, then put `typedef` in front. The variable's name becomes the type's name.

```
typedef int   BOOL;          // like  int BOOL;       -> BOOL is another name for int
typedef char  name_t[100];   // like  char name_t[100]; -> name_t is the type "array of 100 chars"
typedef char *Pointer;       // like  char *Pointer;  -> Pointer is the type "char *"
```

3.3 Operations on structs [7:16-17]

- **Assignment** (`a = b`) copies **all members**.
- A struct can be **passed to a function by value**, even if some members are arrays (the whole array inside the struct is copied).
- A struct can be **returned** from a function.
- If you pass by value, the function works on a copy, so it **cannot change the caller's members**.
- Passing or returning large structures is **costly** (everything is copied). **Use pointers to structures!**

Slide 16

```
TPerson a, b, c;
...
a = b;                       // copies every member of b into a
c = searchPerson("name");    // returning a struct by value
```

Pass by reference [7:17]: pass a pointer. Use `->` to access members through a pointer.

```
typedef struct Person {
    char name[32];
    int  age;
    char gender;
} TPerson;

TPerson * init_Person(TPerson *p, char *name, int age, char gender)
{
    strcpy(p->name, name);     // same as  strcpy((*p).name, name)
    p->age    = age;           // same as  (*p).age = age
    p->gender = gender;        // same as  (*p).gender = gender
    return p;
}
```

- `p->age` is shorthand for `(*p).age`. The parentheses are needed in the long form because `.` binds tighter than `*`.
- Returning `p` lets callers chain calls or initialise inline. It needs `#include <string.h>` for `strcpy`.

Full demo [7:35] (value semantics in action):

```
TPerson init(char name[], int age, char gender) {   // returns a struct BY VALUE
    TPerson p;
    int i;
    for (i = 0; name[i] > 0; i++)     // copy chars until the '\0'
        p.name[i] = name[i];
    p.name[i] = '\0';                 // terminate the string
    p.age = age;
    p.gender = gender;
    return p;                         // a copy of p goes back to the caller
}

void print_info(TPerson p) {          // gets a COPY of the struct
    printf("name: %s, age: %d, gender: %c\n", p.name, p.age, p.gender);
}

int main() {
    TPerson family[4];
    family[0] = init("Alice",   34, 'F');
    family[1] = init("Bob",     40, 'M');
    family[2] = init("Charles", 15, 'M');
    family[3] = init("David",   13, 'M');
    print_info(family[0]);  print_info(family[1]);
    print_info(family[2]);  print_info(family[3]);
    family[1] = family[0];            // struct assignment: Bob's slot becomes a copy of Alice
    print_info(family[1]);            // name: Alice, age: 34, gender: F
    return 0;
}
```

Output: the four people in order, then a fifth line `name: Alice, age: 34, gender: F` after `family[1] = family[0]`. After the assignment `family[1]` and `family[0]` hold equal but **independent** copies, because the whole struct (including the `name` array) is copied.

> **Beyond the slides (added for understanding)**
>
> `init` has no bounds check: a name of 32 or more characters overflows `p.name`. The loop condition `name[i] > 0` also stops at any char with a negative value if `char` is signed; `name[i] != '\0'` is the usual form.

3.4 Self-referential structures [7:24-25]

- A struct **cannot contain a copy of itself** (it would have infinite size), but it **can contain a pointer to its own type**. This is the basis for lists and trees.

Slide 24

```
struct Person {
    int   age;
    char  gender;
    char  name[32];
    struct Person * parents;      // a pointer to this type of struct
} person1, person2;               // variables can be defined right here
```

Mutually referring structs [7:25]: `student` can point to a `book` that is defined *later*, because a pointer to an incomplete type is fine.

```
struct student {
    char  name[128];
    // pointer to a struct defined later is OK.
    // But you CANNOT define an array of book here (e.g. books[8]):
    // the compiler does not know the size of struct book yet.
    struct book * books;
};

struct book {
    char  title[128];
    struct student * owner;
    struct book    * next;        // pointer to this type of struct
};
```

The compiler needs to know a member's **size** to lay out a struct. A pointer is always the same size, so a pointer to a not-yet-defined struct is fine; an array or a plain member of that type is not.

Part 4 - Alignment and padding

4.1 Memory alignment requirements [7:33]

- Memory used to store a value of type X **must be lined up on a multiple of the natural alignment for X**.
- **Why?** Performance (the hardware fetches aligned data efficiently).
- If you do not respect alignment requirements you can get a **bus error (`SIGBUS`)**. The OS will kill your program.

Type	Alignment on x64	Meaning
char	1	any address
short	2	address divisible by 2
int	4	address divisible by 4
long	8	address divisible by 8
float	4	address divisible by 4
double	8	address divisible by 8

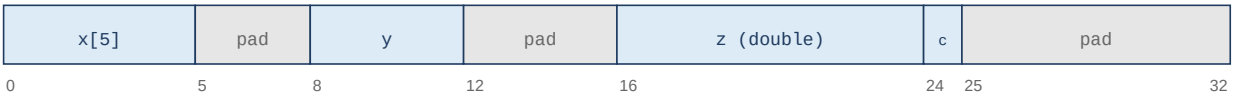
4.2 Structure alignment [7:18]

Members are aligned to their natural type, so the compiler inserts **padding** bytes between members when needed, and also at the end of the struct.

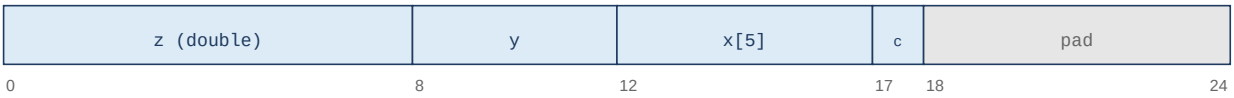
```
struct struct_random {         // members in the 'unlucky' order
    char    x[5];     // bytes 0-4
    int     y;        // bytes 8-11    (needs a multiple of 4  -> pad 5,6,7)
    double  z;        // bytes 16-23   (needs a multiple of 8  -> pad 12-15)
    char    c;        // byte 24
};                    // Total size 32 (pad 25-31 so the size is a multiple of 8)

struct struct_sorted {         // largest alignment first
    double  z;        // bytes 0-7
    int     y;        // bytes 8-11
    char    x[5];     // bytes 12-16
    char    c;        // byte 17
};                    // Total size 24 (18 bytes of data, rounded up to a multiple of 8)
```

struct_random: 32 bytes (grey = padding)



struct_sorted: 24 bytes



Beyond the slides (added for understanding)

The slide gives the totals (32 and 24). The reason for the **trailing** padding is that the struct's size must be a multiple of its largest member alignment (8 because of the `double`), so that every element of an array of these structs stays aligned. In the sorted version, 18 bytes of data are rounded up to 24.

Rule of thumb: declare members from largest alignment to smallest to waste less space.

4.3 Good news and bad news [7:34]

- **Good news:** the compiler handles alignment automatically about 99% of the time.
- **Bad news:** programmers handle the rest:
  - when you do pointer arithmetic (obviously)
  - **do not assume where the fields are**. You must understand the layout (and padding) of structures to compute addresses inside them correctly (slide note)
  - when you call system routines with specific alignment needs, your arguments must comply
  - compiler annotations can force specific alignments (beyond our scope, but remember they exist)

Watch out

Never compute a field's address by hand-adding sizes (`base + sizeof(char[5]) + sizeof(int)`...). Padding makes that wrong. Use `&s.y` or `&p->y` and let the compiler do it.

Part 5 - Returning more than one value [7:22]

A C function returns only one value. The slide lists the ways around that:

Technique	Example from the slide	Notes
Use references (pointers): the **caller prepares the storage**	`long int strtol(const char *str, char **endptr, int base);`	The function writes the extra result through `endptr`.
Use arrays (caller prepares the storage)	`int pipe(int pipefd[2]);`	The function fills `pipefd[0]` and `pipefd[1]`.
Return a structure	-	Costly if the structure is large.
Return a pointer (to array or structure)	`char *strdup(const char *str1);` `struct tm *localtime(const time_t *time);`	The memory must be dynamically allocated or static. **RTM** (read the manual) to see who frees it.
Use global variables	`errno`	**DON'T** (except as the library already does).

Beyond the slides (added for understanding)

Never return a pointer to a normal local variable: it disappears when the function returns. `strdup` returns malloc'd memory (you must `free` it); `localtime` returns a pointer to static storage (it will be overwritten by the next call).

Part 6 - Linked lists (SPC8)

6.1 The node and the head [8:2-3]

- A linked list is **a chain of nodes**. Starting from `head`, each node has a reference (pointer) to the next node.
- The last node's `next` is `NULL`.

Slide 2

```
typedef struct node_tag {
    int    v;                 // data
    struct node_tag * next;   // pointer to this type of struct
} node;                       // define a type, easier to use
```

Inside the struct you must write `struct node_tag *` because the typedef name `node` does not exist yet at that point. After the closing brace, you can use the short name `node`.

Slide 3

```
node * head;       // head is a POINTER, not a node!
head = NULL;       // at the beginning the list is empty
```



> **Key idea**
>
> `head` is only a pointer to the first node. An **empty list is** `head == NULL`. Because the head can change, functions that add or remove nodes at the front must **return the new head**.

6.2 Create a node [8:4]

```
node * new_node(int v)               // create a node for value v
{
    node *p = malloc(sizeof(node));  // allocate memory
    assert(p != NULL);               // you can be nicer

    p->v = v;                        // set the value (or (*p).v)
    p->next = NULL;                  // not linked to anything yet
    return p;
}
// Q on the slide: is it similar to creating objects using "new"?
```

Answer to the slide's question: yes, conceptually. `new_node` plays the role of a constructor: it allocates memory on the heap and initialises the fields, and the caller receives a reference to it. Unlike Java, nothing frees it automatically: you must `free` it yourself.

> **Beyond the slides (added for understanding)**
>
> `assert` needs `#include <assert.h>`, and `malloc` needs `<stdlib.h>`. Always initialise `next` (here to `NULL`); an uninitialised `next` is a classic crash.

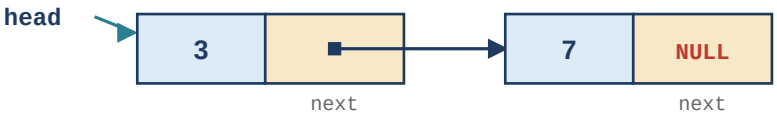
6.3 Prepend (insert at the front) [8:5-6]

The slide first asks "How?", then shows the answer. Work it out: the new node should point at the old first node, and then become the new head.

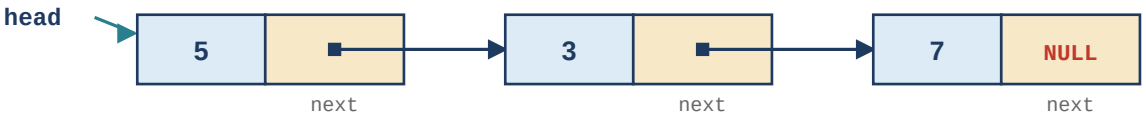
```
node * prepend(node * head, node * newnode)
{
    newnode->next = head;     // works even if the list is empty (head is NULL)
    return newnode;           // head changed !!
}
```

- Empty list: `newnode->next = NULL`, and the new node becomes the whole list.
- Non-empty list: `newnode->next` points at the old head; the new node is returned as the new head.
- **Cost: O(1)**. No traversal is needed.
- Caller usage: `head = prepend(head, new_node(5));`. You **must** assign the result back, or the caller's `head` still points at the old first node.

Before:



After `head = prepend(head, new_node(5));`



6.4 Find the last node [8:7-8]

```
node * find_last(node * head)
{
    if (head != NULL) {                 // only if the list is not empty
        while (head->next != NULL)      // stop ON the last node, not after it
            head = head->next;          // walk forward
    }
    return head;                        // NULL for an empty list
}
```

- The `if` protects against an empty list: `head->next` on a `NULL` pointer would crash.
- The loop condition is `head->next != NULL` (not `head != NULL`), so the loop **stops on the last node** instead of running past it.
- `head` here is a local copy of the pointer, so moving it does not change the caller's head.
- **Cost: O(n)** (must walk the whole list).

6.5 Append (insert at the end) [8:9-10]

```
node * append(node * head, node * newnode)
{
    node *last = find_last(head);   // find the last one
    if (last == NULL)               // if the list is empty, newnode is the head
        return newnode;
    last->next = newnode;           // link the old last node to the new one
    newnode->next = NULL;           // new node is now the last
    return head;                    // return the (unchanged) head
}
```

- Two cases: **empty list** (the new node *is* the list, so the head changes) and **non-empty list** (link after the last node; the head is unchanged).
- It reuses `find_last`, so append is **O(n)**. Prepend is O(1), which is why prepend is the cheaper way to build a list when order does not matter.

Function	Head changes?	Empty-list case	Cost
`prepend`	Always (returns newnode)	handled automatically	O(1)
`find_last`	No	returns NULL	O(n)
`append`	Only if list was empty	returns newnode	O(n)

```
node *head = NULL;
head = append(head, new_node(3));
head = append(head, new_node(7));
head = prepend(head, new_node(1));      // list: 1 -> 3 -> 7
```

> **Beyond the slides (added for understanding)**
>
> Not on the slides, but you will need it: free a list by saving `next` before freeing each node.
>
> while (head) { node *nx = head->next; free(head); head = nx; }

Part 7 - Enumeration types [8:11-12]

- User-defined **integer-like** types.
- The names look like C identifiers and are **listed (enumerated) in the definition**.
- They are **treated as integers**: you can add, subtract and compare them (`Red + Green`, `if (color == Red)`).
- You **can't print them as symbols** (printing gives the number), but the debugger generally shows the name.

```
typedef enum {
    Red, Orange, Yellow, Green, Blue, Violet
} Color;                                    // Red = 0, Orange = 1, ... Violet = 5
```

Numbering [8:12]: values start at **0 by default**, and a value can be set explicitly, after which the next names continue counting up.

```
enum week { Sun, Mon, Tue, Wed, Thur, Fri, Sat };   // Sun=0, Mon=1, ... Sat=6
enum week dow = Mon;

enum status { OK = 1, Warning, Error, Fatal };      // OK=1, Warning=2, Error=3, Fatal=4
```

Enums replace "magic numbers" with readable names (`if (s == Error)` instead of `if (s == 3)`).

Part 8 - The const qualifier [8:13]

`const` says "this may not be modified". With pointers there are **two things** that could be constant: the pointed-to data and the pointer itself. Which one is const depends on where `const` sits relative to the `*`.

```
const int  a = 10;              // constant int: cannot change a
int        b = 5;               // (a normal int, used in the next lines)

const int *pa = &a;             // pointer to const int: change pa yes, *pa no
int * const pb = &b;            // const pointer to int: change *pb yes, pb no
const int * const pc = &a;      // const pointer to const int: neither

char * strcpy(char * dest, const char * src);   // cannot change source
```

Declaration	Can change the pointer?	Can change the data?
`const int *pa`	yes	no
`int * const pb`	no	yes
`const int * const pc`	no	no

> **Memory trick (not on the slide)**
>
> **Trick for reading:** read the declaration right to left. `int * const pb` = "pb is a constant pointer to int". `const int *pa` = "pa is a pointer to an int that is const". Or: whatever is directly to the **right of** `const` is what cannot change (`* const` -> the pointer; `const int` -> the int).

The C99 wording in the slide notes: the contents of an object pointed to by a `ptr_to_constant` shall not be modified *through that pointer*, but the pointer itself may be changed to point to another object. Similarly, for a constant pointer the pointed-to int may be modified, but the pointer always points to the same place.

Why it matters: in `strcpy(dest, src)`, `const char *src` is a promise to callers that the function will not modify the source string, and it lets you pass string literals and const data safely.

Part 9 - Function pointers

9.1 Declaring function pointers [8:14]

```
int    func();          /* a function returning an integer                    */
int *  func();          /* a function returning a pointer to an integer       */
int    (*func)();       /* a POINTER TO a function returning an integer       */
int *  (*func)();       /* a pointer to a function returning a pointer to int */
```

- The parentheses around `(*func)` are what turn "function" into "pointer to function". Without them, `*` binds to the return type.
- Compare `int *func()` (function that returns `int *`) with `int (*func)()` (pointer to a function that returns `int`).

9.2 Example [8:15]

```
int mymax(int a, int b)
{
    return (a > b) ? a : b;
}

int (*pf)(int a, int b);     // pf is a pointer to a function taking (int,int) returning int

pf = mymax;                  // assign: C99 style. Note that it is NOT  mymax()
pf(3, 5);                    // call through pf  -> 5

pf = &mymax;                 // also valid: & is optional for functions
(*pf)(3, 5);                 // also valid: explicit dereference is optional
```

- `mymax` (no parentheses) is the function's **address**; `mymax()` would *call* it and give you an `int`.
- All of `pf = mymax`, `pf = &mymax`, `pf(3,5)` and `(*pf)(3,5)` are valid and equivalent forms.

9.3 What they are used for [8:16]

- **Call-back mechanisms**: you hand a function to someone else's code to be called later.
- **Generic functions** (see `qsort` below).
- `pthread_create()` (takes the function the new thread should run), **dynamic signal handlers**, and more.
- You can store function pointers **in arrays**, and arrays **in structures**.
- You can **simulate objects** of object-oriented languages: a struct holding data plus pointers to the functions that operate on it.

9.4 qsort, the C library quicksort [8:17-22]

```
void qsort(void * base,
           size_t nel,
           size_t width,
           int (*compare)(const void *, const void *));        // in <stdlib.h>
```

Parameter	Meaning
`base`	address of the array, as an **untyped pointer** (`void *`)
`nel`	the **number of elements** in the array
`width`	the **size in bytes of ONE element**
`compare`	a **pointer to a function** that compares two elements

> **Key idea**
>
> `qsort()` only knows it has been asked to sort `nel` items of `width` bytes each. It does **not** know the element type, or how to compare them. That is exactly why you pass a comparator.

Why pass a function? [8:18] We have **one** generic quicksort implementation and do not want to write a separate one for each data type. Whenever `qsort` needs to rank two elements, it **calls your comparator** with the **addresses** of the two items. The comparator returns:

Return value	Meaning
0	`*a` equals `*b`
positive	`*a` is greater than `*b` (a goes after b)
negative	`*a` is less than `*b` (a goes before b)

Comparators for numbers [8:19]

```
int compare_int(const void *a, const void *b)
{   // qsort() does not know the type, but you know
    return *(int *)a - *(int *)b;
}

int compare_double(const void *a, const void *b)
{
    double va, vb;
    va = *(double *)a;   vb = *(double *)b;
    return va > vb ? 1 : (va < vb ? -1 : 0);
}
```

- The parameters are `const void *`, so you must **cast to the real type** (`(int *)a`) and then **dereference** (`*`) to get the value.
- `compare_double` cannot just subtract: the difference of two doubles may be a small fraction that truncates to 0 when converted to `int`, so it uses explicit comparisons returning 1, -1 or 0.

> **Beyond the slides (added for understanding)**
>
> `*(int*)a - *(int*)b` can **overflow** for very large positive/negative ints (for example `INT_MAX - (-1)`), giving the wrong sign. The safe form is `(x > y) - (x < y)`.

Sorting an array of strings [8:20-22]

Here the elements of the array are themselves **pointers to strings** (`char *`).

```
char **words = malloc(sizeof(char*) * n);     // array of n pointers to strings

words[0] --> "Hello"
words[1] --> "Bob"
   ...
words[n-1] --> "last"
```

- When `qsort` compares `words[0]` and `words[1]`, it passes `&words[0]` **and** `&words[1]` (slide notes). These are the addresses of the *elements*, not of the strings.
- Each element has type `char *`, so the address of an element has type `char **` (a pointer to a `char *`).
- Therefore the comparator must **cast to** `char **`, **dereference once** to get the `char *` string, and then compare the strings with `strcmp`.

Slide 21

```
int compare_string(const void *a, const void *b)
{
    char *s1, *s2;
    s1 = *(char **)a;        // a is (char **): dereference once -> the string
    s2 = *(char **)b;
    return strcmp(s1, s2);   // library function does the string comparison
}

// or on one line
int compare_string(const void *a, const void *b)
{
    return strcmp(*(char**)a, *(char**)b);
}
```

Calling qsort [8:22]:

```
int some_function(void)
{
    ...
    char **words = malloc(sizeof(char*) * n);
    ...
    qsort(words, n, sizeof(char*), compare_string);
    ...
}
```

- `width` is `sizeof(char*)` (the size of one **element**, a pointer), **not** the length of a string.
- `compare_string` is passed **without parentheses**: you are passing the function's address, not calling it.
- The slide's key reminder: **type cast to `char **` before dereferencing.**

> **Watch out**
>
> Why not `strcmp(a, b)` directly? Because `a` and `b` point at the array **slots** (which hold pointers), not at the characters. `strcmp` would treat the pointer bytes as if they were text.

Array element type	qsort `width`	Comparator casts to
`int`	`sizeof(int)`	`const int *` / `(int *)`
`double`	`sizeof(double)`	`(double *)`
`char *` (strings)	`sizeof(char *)`	`(char **)`, then `strcmp`
`struct Person`	`sizeof(struct Person)`	`(struct Person *)`, compare a member

Part 10 - Cheat sheet and self-check

10.1 One-page cheat sheet

Topic	Rule to remember
`p + k`	address + `k * sizeof(*p)`. Type stays `T *`.
`p - q`	number of **elements** between them (same type needed).
`p < q`	compares addresses as unsigned integers.
`(char *)p + n`	adds exactly n **bytes** (cast first, then add).
`a[i]`	same as `*(a+i)` and `i[a]`.
array decay	array -> pointer to element 0, except under `sizeof`, `&`, string-literal init.
`sizeof(a)` vs `sizeof(p)`	40 vs 8 for `int a[10]` and `int *p` on 64-bit. Parameters are pointers.
`a++` on array	compile error. `p++` on pointer is fine.
struct assign / pass	copies **all** members (even arrays inside). Use pointers for big structs.
`p->x`	same as `(*p).x`.
struct size	members padded to natural alignment; total rounded up to largest alignment.
`void *` arithmetic	not standard C; gcc treats sizeof(void) as 1.
list head	a pointer, `NULL` = empty. Return the new head from prepend/append.
prepend / append	O(1) / O(n) (append walks to the end).
enum	ints starting at 0 unless set; later names continue counting.
const placement	`const int *p` data fixed; `int * const p` pointer fixed.
function pointer	`int (*pf)(int,int) = mymax;` call as `pf(3,5)`.
qsort comparator	gets addresses of elements as `const void *`: cast, then dereference.

10.2 Common bugs

- Forgetting that `p + 1` moves by `sizeof(*p)` bytes, not 1.
- Using `sizeof(p)` on a pointer parameter and expecting the array size.
- Not assigning back the return value of `prepend`/`append` (the head is lost or stale).
- Dereferencing `head->next` when `head` is `NULL`.
- Never `free`ing `malloc`'d nodes, or using a node after freeing it.
- Forgetting to NUL-terminate strings or exceeding `name[32]` (31 characters + `'\0'`).
- Passing `strcmp` the array slots (`char **`) instead of the strings (`char *`) in a qsort comparator.
- Calling a function pointer assignment with parentheses: `pf = mymax();` calls the function.
- Assuming struct fields are packed with no padding, or hand-computing field addresses.

10.3 Self-check questions

Q1. If `int *p` holds 2000, what are `p + 3` and `p - 1` (4-byte ints)?

Answer: 2012 and 1996.

Q2. `int a[10]; int *p = a;`. What are `sizeof(a)`, `sizeof(p)`, `sizeof(&a)` on a 64-bit machine?

Answer: 40, 8, 8 (`&a` is a pointer to the whole array).

Q3. What is the difference between `(char *)t + 8` and `(char *)(t + 8)` for `int *t`?

Answer: 8 bytes versus 32 bytes past `t`.

Q4. Why does `a++` fail to compile for an array `a`?

Answer: An array name is not a modifiable lvalue; it is like incrementing a constant.

Q5. Write `*p++ = i;` as three statements.

Answer: `int *tp = p; p++; *tp = i;`

Q6. Total size of `struct { char c; int i; char d; }` on x64?

Answer: 12 bytes: c at 0, pad 1-3, i at 4-7, d at 8, pad 9-11. Reordering to `int, char, char` gives 8.

Q7. Why must `prepend` return a node pointer?

Answer: The head changes, and the caller only has its own copy of the old `head` pointer.

Q8. What are the values in `enum status {OK = 1, Warning, Error, Fatal};`?

Answer: OK=1, Warning=2, Error=3, Fatal=4.

Q9. Which of `const int *p`, `int * const p` lets you do `p++`?

Answer: `const int *p` (only the data is const).

Q10. In `compare_string`, why is the cast `(char **)` and not `(char *)`?

Answer: The comparator receives the address of an element, and each element is itself a `char *`.

> **Note on the slides**
>
> SPC7 slide 19 says "Study the remaining slides yourself. Also study the demo code!". The slides after it (array copying, pointer casting, returning multiple values, typedef, self-referential structs, the `tab+3` worked example, alignment, and the full Person demo) are all covered in these notes, so they are fair game for quizzes and exams. Also run and modify the demo code from class.